

PEC 2 PROJECT

Academic Year: 2025-2026

Year: TY B.Tech

Semester: II

Title: Design and implement a System Verilog testbench to verify the functionality of a FIFO

Objectives of the Expt.:

- To design a synchronous FIFO memory buffer using SystemVerilog.
- To simulate the design using EDA Playground and observe read/write operations.

Equipment / Software Required: EDA Playground

Theory:

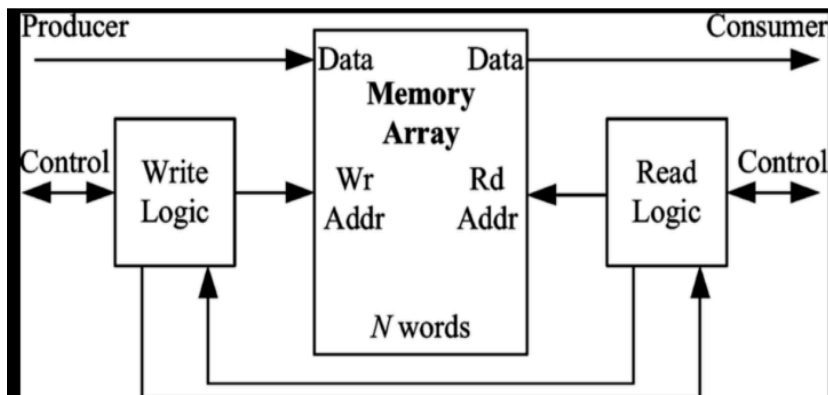
A FIFO (First-In First-Out) is a type of data buffer or queue in which the first piece of data written into the buffer is the first to be read out. FIFOs are fundamental building blocks in digital systems, widely used for data buffering, rate matching, and inter-module communication. In digital design, two modules often operate at different clock speeds or data rates. A FIFO provides a temporary storage mechanism that ensures no data is lost and data integrity is maintained across the interface.

A FIFO operates using two independent pointers: a write pointer (Wr Addr) and a read pointer (Rd Addr). The write pointer advances each time data is written into the memory, and the read pointer advances each time data is read. Two status flags are critical to FIFO operation:

- FULL flag: Asserted when the FIFO is completely filled and no more data can be written.
- EMPTY flag: Asserted when the FIFO contains no data and no read operation is possible.

The FIFO design must handle boundary conditions carefully to prevent overflow (writing to a full FIFO) and underflow (reading from an empty FIFO). Functional verification using SystemVerilog ensures that the FIFO behaves correctly before hardware fabrication.

Block Diagram / Architecture of FIFO:



The FIFO architecture consists of the following components:

1. Producer (Input Source)

- The producer sends data into the FIFO.
- This data is written into the memory through the write logic.

2. Write Logic

- Controls the write operation.
- Uses the write address (Wr Addr) to store incoming data in the memory array.

3. Data Memory Array

- This is the FIFO storage buffer.
- It stores N words of data in the order they are received.

4. Read Logic

- Handles the read operation.
- Uses the read address (Rd Addr) to fetch the oldest data from memory.

5. Consumer (Output)

- Receives data from the FIFO.
- Data comes out in the same order it was written (First-In First-Out).

Verification Goals:

- Ensure data is read in the same order it was written (FIFO behavior).
- Verify correct data transfer between input and output.
- Detect overflow condition — writing when FIFO is full.
- Detects underflow condition — reading when FIFO is empty.
- Verify correct update of read and write pointers.

Operation Table for FIFO:

write_en	read_en	full	empty	Operation
1	0	0	X	Write data_in to FIFO; count++
0	1	X	0	Read data_out from FIFO; count--
1	1	0	0	Simultaneous Read & Write; count unchanged
1	0	1	0	FIFO Full — write ignored (overflow)

0	1	X	1	FIFO Empty — read ignored (underflow)
0	0	X	X	No operation; FIFO idle

Design Code:

```

module fifo #(
    parameter DATA_WIDTH = 8,
    parameter DEPTH = 8
)
(
    input  logic clk,
    input  logic rst,
    input  logic write_en,
    input  logic read_en,
    input  logic [DATA_WIDTH-1:0] data_in,
    output logic [DATA_WIDTH-1:0] data_out,
    output logic full,
    output logic empty
);

// Memory
logic [DATA_WIDTH-1:0] mem [0:DEPTH-1];

// Pointers and count
logic [$clog2(DEPTH)-1:0] write_ptr;
logic [$clog2(DEPTH)-1:0] read_ptr;
logic [$clog2(DEPTH):0]  count;

// Status flags
assign full  = (count == DEPTH);
assign empty = (count == 0);

// FIFO logic
always_ff @(posedge clk or posedge rst) begin
    if (rst) begin
        write_ptr <= 0;
        read_ptr  <= 0;
        count     <= 0;
        data_out  <= 0;
    end else begin

```

```

// Write logic
if (write_en && !full) begin
    mem[write_ptr] <= data_in;
    write_ptr <= (write_ptr + 1) % DEPTH;
end

// Read Logic
if (read_en && !empty) begin
    data_out <= mem[read_ptr];
    read_ptr <= (read_ptr + 1) % DEPTH;
end

case ({write_en && !full, read_en && !empty})
    2'b10: count <= count + 1; // write only
    2'b01: count <= count - 1; // read only
    2'b11: count <= count;    // simultaneous → no change
    default: count <= count;
endcase
end
end
endmodule

```

Explanation of Design Code:

1. The module fifo is parameterized with DEPTH = 8 (number of storage locations) and DATA_WIDTH = 8 (bit-width of each word).
2. The inputs include clk (clock), rst (reset), wr_en (write enable), rd_en (read enable), and data_in (8-bit write data).
3. The outputs include data_out (8-bit read data), full (FIFO is completely filled), and empty (FIFO contains no data).
4. Memory array mem[0:DEPTH-1] stores data elements in the order they are received.
5. wr_ptr is the write pointer that increments with each write, and rd_ptr is the read pointer that increments with each read.
6. The full flag is asserted when count == DEPTH, and the empty flag is asserted when count == 0.
7. On receiving rst, all pointers and counters are reset to zero, ensuring a clean initial state.
8. A write operation is only performed if wr_en = 1 and the FIFO is not full; similarly, a read is only performed if rd_en = 1 and the FIFO is not empty.
9. Case Statement ensures the count is updated correctly for write-only, read-only, and simultaneous read-write scenarios.
10. Non-blocking assignments (<=) are used throughout the always_ff block to model synchronous flip-flop behaviour correctly.

Testbench:

A testbench is a simulation program used to verify the functional correctness of a digital design. It does not represent actual hardware. It generates stimulus signals such as clock, reset, write enable, read enable, and data inputs, then observes the DUT outputs and checks them against expected values.

Testbench for FIFO:

```
`timescale 1ns/1ps
```

```
module tb;
```

```
    parameter DEPTH = 8;
```

```
    parameter DATA_WIDTH = 8;
```

```
    // DUT signals
```

```
    logic clk, rst;
```

```
    logic write_en, read_en;
```

```
    logic [DATA_WIDTH-1:0] data_in;
```

```
    logic[DATA_WIDTH-1:0] data_out;
```

```
    logic full, empty;
```

```
    // Instantiate DUT
```

```
    fifo #(DATA_WIDTH, DEPTH) dut (
```

```
        .clk(clk),
```

```
        .rst(rst),
```

```
        .write_en(write_en),
```

```
        .read_en(read_en),
```

```
        .data_in(data_in),
```

```
        .data_out(data_out),
```

```
        .full(full),
```

```
        .empty(empty)
```

```
    );
```

```
    // Clock
```

always #5 clk = ~clk; //clock toggles every 5 nanoseconds,time period is 10ns.

// Scoreboard

logic [DATA_WIDTH-1:0] model_q[\$]; //generate expected o/p

logic pending_read;

logic [DATA_WIDTH-1:0] pending_expected;

//Reset

task reset();

rst = 1; write_en = 0; read_en = 0; data_in = 0;

pending_read = 0;

repeat(2) @(posedge clk);

rst = 0;

endtask

initial begin

\$dumpfile("dump.vcd");

\$dumpvars(0, tb); //Stores all signals of testbench into waveform file.

clk = 0;

reset();

repeat(100) begin @(posedge clk);

#1; // small delay to let signals settle after clock edge

// Step 1: Check previous cycle's read result NOW

if (pending_read) begin

if (data_out !== pending_expected)

\$error("MISMATCH: expected=%0d got=%0d", pending_expected, data_out);

pending_read = 0;

end

// Step 2: Apply new stimulus

write_en = \$urandom_range(0, 1);

read_en = \$urandom_range(0, 1);

data_in = \$urandom;

```

// Step 3: Update model for this cycle's actions
if (write_en && !full)
    model_q.push_back(data_in); //built-in SystemVerilog queue methods.
if (read_en && !empty) begin
    pending_expected = model_q.pop_front();
    pending_read = 1;          // check data_out on NEXT clock edge
end
end

// Check last pending read if any
if (pending_read) begin
    @(posedge clk); #1;
    if (data_out !== pending_expected)
        $error("MISMATCH: expected=%0d got=%0d", pending_expected, data_out);
end

$display("Test Passed");
$finish;
end
endmodule

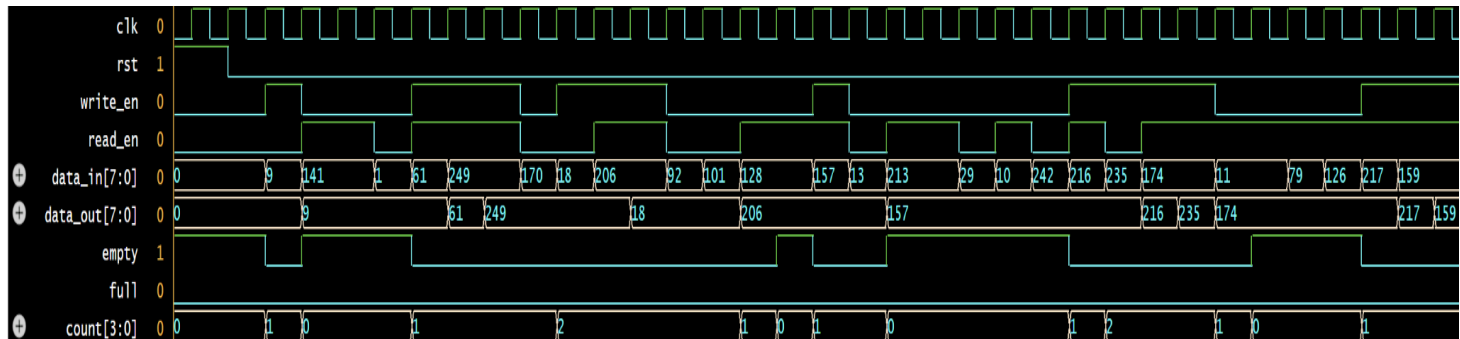
```

Explanation of Testbench:

1. The testbench module tb instantiates the FIFO DUT with DATA_WIDTH = 8 bits and DEPTH = 8 locations.
2. The clock is generated using always #5 clk = ~clk, producing a clock period of 10 ns.
3. The reset task asserts rst for 2 clock cycles to initialize all internal pointers and count register to zero, then de-asserts it to begin operation.
4. \$dumpfile and \$dumpvars generate a .vcd waveform file that can be viewed using GTKWave on EDA Playground.
5. write_en and read_en are randomized using \$urandom_range(0,1) to generate write-only, read-only, simultaneous, and idle test scenarios.
6. When a write occurs and FIFO is not full, data_in is pushed into model_q using push_back() to record the expected output order.
7. When a read occurs and FIFO is not empty, the expected value is popped from model_q and compared against data_out on the next clock edge.

8. If a mismatch is detected, \$error reports the failure. On successful completion, \$display prints "Test Passed" and \$finish ends the simulation.

Output:



The simulation output on EDA Playground shows:

1. **Clock Signal (clk):** Provides synchronous timing to all flip-flops. The FIFO state changes only at the rising clock edge.
2. **Reset Signal (rst):** When asserted, all internal registers (write_ptr, read_ptr, count, data_out) are cleared to zero. Normal operation resumes once rst is de-asserted.
3. **Write Operation:** When write_en = 1 and full = 0, data_in is stored in the memory array at write_ptr. The write_ptr advances and count increments by 1.
4. **Read Operation:** When read_en = 1 and empty = 0, the data at read_ptr is transferred to data_out. The read_ptr advances and the count decreases by 1.
5. **Full Flag:** Observed to assert (full = 1) when count reaches DEPTH (8). Further writes are suppressed to prevent overflow.
6. **Empty Flag:** Observed to assert (empty = 1) when count reaches 0. Further reads are suppressed to prevent underflow.
7. **FIFO Ordering:** The waveform confirms that data_out matches data_in values in the same order they were written, validating the First-In First-Out property.

Applications:

1. **Clock Domain Crossing (CDC):** FIFOs safely transfer data between modules running on different clocks.
2. **Network Packet Buffering:** FIFOs are used in routers and switches to buffer incoming packets.
3. **Processor Pipelines:** Instruction FIFOs store instructions between pipeline stages.
4. **UART and Serial Communication:** Transmit and receive FIFOs buffer serial data to prevent loss during high-speed transfer.
5. **SoC Design:** FIFOs interface AXI and AHB bus masters/slaves with different throughput rates.

Conclusion:

In this experiment, a synchronous FIFO buffer was successfully designed and implemented using SystemVerilog. The FIFO correctly stored and retrieved 8-bit data in First-In First-Out order with parameterizable depth. A comprehensive verification environment was developed, including a randomized stimulus generator, driver, golden-reference scoreboard (model_q queue), and mismatch checker. The simulation confirmed correct operation under all tested scenarios: normal writes and reads, simultaneous read-write, overflow prevention (write blocked when full), and underflow prevention (read blocked when empty). The full and empty flags were verified to assert and de-assert correctly in sync with the count register. This experiment provided practical understanding of FIFO design, SystemVerilog randomization, and functional verification methodology.

References:

- I. Stuart Sutherland, Simon Davidmann, Peter Flake, *SystemVerilog for Design*, Springer.
- II. Chris Spear, Greg Tumbush, *SystemVerilog for Verification*, Springer.
- III. Janick Bergeron, *Writing Testbenches: Functional Verification of HDL Models*, Springer.
- IV. IEEE Standard for SystemVerilog (IEEE 1800), *SystemVerilog Language Reference Manual*.
- V. EDA Playground, *Online Verilog/SystemVerilog Simulator*, <https://www.edaplayground.com>